



Bilkent University

CS 319

Deliverable-4

S1-T2-Databaseable

Hüsnü Arkca

Yunus Emre Aslan

Mehmet Dizdar

Aral Müftüoğlu

Onur Ege Yakar

1. Design Goals

1.1 Usable for diverse stakeholders

The interface should be easy to learn for researchers, participants, admins, and reviewers, supporting flows like study creation, artifact upload, quizzes, dashboards, and evaluation with clear navigation and feedback. The main tradeoff is limiting very complex customization options in order to keep the UI consistent and simple.

1.2 Data quality, security, and ethics

The platform needs a strong role-based access control, secure authentication, and privacy-preserving features such as blinded evaluations while reliably storing ratings, annotations, and tool metrics for later analysis. These protections may add overhead to implementation and performance, but they are essential for credible human-subject studies.

1.3 Performance and scalability for large studies

The system should remain responsive while handling large artifacts, multiple concurrent users, dashboards, and repeated comparisons, including bulk uploads and tool outputs. To achieve this, some advanced features (for example, very heavy real-time visualizations) may be simplified or deferred to keep the core flows fast and robust.

1.4 Testability and deployability

Critical workflows (registration, study creation, quizzes, artifact evaluation, export) should be easy to test automatically and to deploy both locally (e.g., via a single script or container) and on a shared server. This motivates clear boundaries and configuration conventions, even if it means more upfront work on testing and deployment scripts.

1.5 Maintainability and long-term sustainability

The system should be structured in a modular and well-documented way so that it can be easily updated, debugged, and extended as new feature types, tools, or evaluation methods are introduced. Clear coding standards, separation of concerns, and version control practices support long-term maintenance by different.

2.Connectors

Web UI ↔ Application layer: A standard HTTP/REST request–response connector is used between browser clients and the backend, supporting operations such as managing users, uploading artifacts, taking quizzes, performing comparisons, and viewing dashboards. This matches the multi-user web-application requirement and keeps clients simple.

Application layer ↔ Persistence: Repositories or ORM(Object Relational Mapping) connectors encapsulate all access to the database for entities like users, studies, artifacts, evaluations, and tool metrics, so that storage details can evolve without breaking higher layers. This extra indirection slightly increases complexity but supports the requirement for extensible artifact types and analysis results.

Application layer ↔ External tools / LLMs: When generating and grading quiz questions with LLM APIs or importing metrics from analysis tools, the system uses HTTP-based service connectors and file/JSON import connectors. These connectors allow optional integrations while keeping the core system independent of any single external provider.

3.Architectural Style

Layered

The system adopts a layered architecture with at least presentation, application, domain, and infrastructure layers.
(DB-Repository-Service-Controller-Frontend) This style directly supports the course requirement for a modular, maintainable, object-oriented design and fits the need to separate concerns such as UI workflows, study/business rules, artifact/evaluation models, and storage or external tool integration.

Compared with alternatives like microservices or event-driven architectures, the layered style is simpler to implement and reason about for a single web application, while still allowing future extension of artifact types, analysis tools, and deployment options specified in the project description. The main tradeoff is some additional call overhead between layers, which is acceptable given the moderate performance requirements and strong emphasis on clarity, testability, and extensibility

